Tutorial 2: MV Effects and Reference Voltage Exercise Solutions

Before you start

In a fresh kernel, run Sections 2.2–2.4 of `Tutorial_2_MV_Effects_Reference_Voltage.ipynb`. Then run the seven listings below in order in the Exercise Workspace. Sections 2.5–2.6 are not needed. The listings use the notebook's existing data and helpers.

Each fit uses 40 epochs, independently of quick/full mode. Select settings and a seed using three training-data folds; leave the test set until E2.3. This is same-time estimation from simulated data, not forecasting. The exercises reuse the tutorial's test period, not a new independent test set.

How to read this document. Each task is followed by its recorded output and a short explanation. Values come from the existing exercise CSV/JSON records and may vary slightly on another run. Selection uses unrounded scores, not the rounded values printed here. File-export code is omitted so the listings focus on the exercise itself.

E1.1: Prepare two input sets and the same targets

Both models use every customer's P/Q. Only the second model also receives the three supplied Vref columns. All customer voltages remain targets. Do not fit scalers yet; the validation helper handles this within each fold.

```
e1_reference_columns = REFERENCE_PHASES.copy()
e1_target_customers = np.arange(1, C + 1)
e1_model_inputs = {
    "PQ_only": PQ_mv_tr,
    "PQ_plus_Vref": np.column_stack([
        PQ_mv_tr, V_secondary_mv_tr
    ]).astype(np.float32),
}
e1_targets = V_mv_tr
e1_model_names = list(e1_model_inputs)

for name in e1_model_names:
    print(MODEL_LABELS[name], "inputs:", e1_model_inputs[name].shape)
print("Targets:", e1_targets.shape)
print("Reference columns:", e1_reference_columns)
print("Target customers:", e1_target_customers.tolist())
```

Recorded output

Model	Training rows	Inputs	Outputs
P/Q only	6,048	62	31
P/Q + Vref	6,048	65	31

Reference columns: `phase1`, `phase2`, `phase3`. Target customers: 1–31.

The three extra inputs come from `V_secondary.pkl`, not from removing three customer targets. No particular physical measurement node is assumed.

E1.2: Compare 93 and 155 hidden neurons

Use ReLU for both sizes. Keep the learning rate, L2 strength, batch size, epochs, folds and seed unchanged. `run_grid_search()` evaluates both input designs for each size and stores the scores.

```
e1_seed = 42
e1_k = 3
e1_search_space = {
    "hidden_neurons": [93, 155],
    "activation": ["relu"],
    "lr": [1e-3],
    "weight_decay": [1e-5],
    "batch_size": [72],
    "epochs": [40],
}
e1_results = run_grid_search(
    e1_model_inputs, e1_targets, e1_search_space,
    k=e1_k, seed=e1_seed
)
display(e1_results[[
    "hidden_neurons", "PQ_only_RMSE_kfold",
    "PQ_plus_Vref_RMSE_kfold",
]].round(6))
```

Recorded output. Mean validation RMSE, in volts:

Hidden neurons	P/Q only	P/Q + Vref
155	0.584702	0.054287
93	0.593790	0.063779

The 155-neuron network gives a lower mean error for both inputs in this run. E1.3 uses these stored scores; it does not train the networks again.

E1.3: Choose one shared configuration

The existing exercise rule ranks configurations by the *joint score*: the arithmetic mean of the two model-specific mean validation RMSE values. This chooses common settings for the comparison, not separate settings for each model. The individual scores are still reported separately.

```
e1_results = e1_results.sort_values(
    "joint_RMSE_kfold", kind="stable"
).reset_index(drop=True)
display(e1_results[[
    "hidden_neurons",
    "PQ_only_RMSE_kfold", "PQ_only_RMSE_std",
    "PQ_plus_Vref_RMSE_kfold", "PQ_plus_Vref_RMSE_std",
    "joint_RMSE_kfold",
]].round(6))
e1_best_row = e1_results.iloc[0]
e1_best_config = {
    "hidden_neurons": int(e1_best_row["hidden_neurons"]),
    "activation": "relu",
    "lr": 1e-3,
    "weight_decay": 1e-5,
    "batch_size": 72,
    "epochs": 40,
}
print("Selected configuration:", e1_best_config)
```

Recorded output. Fold means and standard deviations are in volts.

Hidden	P/Q mean	P/Q std.	+Vref mean	+Vref std.	Joint
155	0.584702	0.002578	0.054287	0.000410	0.319494
93	0.593790	0.015297	0.063779	0.003495	0.328785

Select **155 hidden neurons with ReLU**. Learning rate 10^{-3}, L2 strength 10^{-5}, batch size 72 and 40 epochs remain unchanged. Standard deviations describe differences across the three folds, not confidence intervals.

E2.1: Choose a shared random seed

Compare seeds 0, 1 and 2 with the selected configuration and the same folds. The returned table is ranked by joint validation score, lowest first. Take the seed from the first row.

```
e2_seed_results = run_seed_search(
    e1_model_inputs, e1_targets, e1_best_config,
    seeds=[0, 1, 2], k=e1_k
)
display(e2_seed_results[[
    "seed", "PQ_only_RMSE_kfold",
    "PQ_plus_Vref_RMSE_kfold", "joint_RMSE_kfold",
]].round(6))
e2_selected_seed = int(e2_seed_results.iloc[0]["seed"])
print("Selected seed:", e2_selected_seed)
```

Recorded output

Seed	P/Q mean RMSE (V)	+Vref mean RMSE (V)	Joint (V)
1	0.597398	0.052285	0.324841
0	0.600967	0.052752	0.326860
2	0.607313	0.054578	0.330946

Selected seed: 1.

This seed has the lowest joint score here. Both final models use it and the selected shared settings. The test set is still unused.

E2.2: Train two fresh models on all training rows

Fit one target scaler and a separate input scaler for each input design using all supplied training rows. Train new models with the selected settings and seed; do not continue training validation-fold models.

```
e2_output_scaler = MinMaxScaler().fit(e1_targets)
e2_Y_train = e2_output_scaler.transform(e1_targets).astype(np.float32)
e2_models = {}
e2_input_scalers = {}
e2_training_rows = []
for name in e1_model_names:
    e2_scaler = MinMaxScaler().fit(e1_model_inputs[name])
    e2_input_scalers[name] = e2_scaler
    e2_X_train = e2_scaler.transform(
        e1_model_inputs[name]
    ).astype(np.float32)
    e2_model = build_model(
        e2_X_train.shape[1], C, e1_best_config,
        seed=e2_selected_seed
    )
    e2_history = train_model(
        e2_model, e2_X_train, e2_Y_train, e1_best_config
    )
    e2_models[name] = e2_model
    e2_training_rows.append({
        "Model": MODEL_LABELS[name],
        "Inputs": e2_model.input_shape[-1],
        "Outputs": e2_model.output_shape[-1],
        "Completed epochs": len(e2_history.epoch),
    })
display(pd.DataFrame(e2_training_rows))
```

Recorded output

Model	Inputs	Outputs	Completed epochs
P/Q only	62	31	40
P/Q + Vref	65	31	40

Both models completed 40 epochs on all 6,048 training rows. Their input sizes differ, but their targets and training budget are the same. These models, not the fold models, are used for the test predictions.

E2.3: Predict test voltages and calculate errors

Use the fitted training scalers to transform the matching test inputs. `predict_unscaled()` restores volts. Calculate overall RMSE/MAE and then one RMSE for each customer, separately for each model.

```
e2_test_inputs = {
    "PQ_only": PQ_mv_te,
    "PQ_plus_Vref": np.column_stack([
        PQ_mv_te, V_secondary_mv_te
    ]).astype(np.float32),
}
e2_test_targets = V_mv_te
e2_predictions = {}
e2_errors = {}
e2_metric_rows = []
for name in e1_model_names:
    e2_X_test = e2_input_scalers[name].transform(
        e2_test_inputs[name]
    ).astype(np.float32)
    e2_predictions[name] = predict_unscaled(
        e2_models[name], e2_X_test, e2_output_scaler
    )
    e2_error = e2_predictions[name] - e2_test_targets
    e2_errors[name] = e2_error
    e2_metric_rows.append({
        "Model": MODEL_LABELS[name],
        "RMSE (V)": np.sqrt(np.mean(e2_error ** 2)),
        "MAE (V)": np.mean(np.abs(e2_error)),
    })
e2_metrics = pd.DataFrame(e2_metric_rows)
# axis=0 averages over time, leaving one value per customer.
e2_customer_rmse = pd.DataFrame({
    "Customer": e1_target_customers,
    "P/Q only RMSE (V)": np.sqrt(np.mean(
        e2_errors["PQ_only"] ** 2, axis=0
    )),
    "P/Q + Vref RMSE (V)": np.sqrt(np.mean(
        e2_errors["PQ_plus_Vref"] ** 2, axis=0
    )),
})
display(e2_metrics.round(6))
display(e2_customer_rmse.round(6))
```

Recorded output. Evaluation uses 2,016 test rows and all 31 customers.

Model	RMSE (V)	MAE (V)
P/Q only	0.658955	0.479735
P/Q + Vref	0.047638	0.034860

The model with Vref has lower test errors in this experiment. RMSE gives more weight to larger errors; MAE is the mean absolute error. Inspect the customer table below as well, rather than relying only on overall scores.

E2.3: Customer-level output

These are the per-customer errors from the same test predictions above. E2.4 compares both models for the worst P/Q-only customer, customer 5.

Table 1: E2.3 output: test RMSE for every customer.

Customer	P/Q only RMSE (V)	P/Q + Vref RMSE (V)
1	0.564115	0.018393
2	0.709498	0.025437
3	0.648981	0.025153
4	0.572872	0.023709
5	0.752155	0.032014
6	0.660002	0.026363
7	0.564298	0.027801
8	0.736837	0.042775
9	0.652373	0.034059
10	0.576141	0.029734
11	0.739425	0.041125
12	0.652856	0.039666
13	0.567449	0.037427
14	0.746576	0.045619
15	0.648879	0.036716
16	0.574454	0.044865
17	0.735714	0.058042
18	0.654817	0.049783
19	0.588425	0.055577
20	0.744757	0.054893
21	0.662171	0.059403
22	0.579683	0.057289
23	0.750198	0.060591
24	0.670091	0.066627
25	0.584982	0.054939
26	0.751607	0.070531
27	0.665785	0.054701
28	0.563807	0.056967
29	0.745268	0.069582
30	0.669573	0.053266
31	0.580494	0.054017

E2.4: Inspect the worst P/Q-only customer

Find the customer with the largest P/Q-only RMSE, then plot that same customer for both models. This is inspection after training, not another selection step. Residuals are estimated voltage minus reference voltage.

```
e2_worst = int(np.argmax(e2_customer_rmse["P/Q only RMSE (V)"]))
e2_customer = int(e1_target_customers[e2_worst])
print("Worst P/Q-only customer:", e2_customer)
display(e2_customer_rmse.iloc[[e2_worst]].round(6))
e2_hours = np.arange(len(e2_test_targets)) * MINUTES_PER_STEP / 60
e2_fig, e2_axes = plt.subplots(2, 1, figsize=(11, 6), sharex=True)
e2_axes[0].plot(
    e2_hours, e2_test_targets[:, e2_worst],
    color="black", label="Simulated reference"
)
for name, colour in [("PQ_only", "tab:red"),
                     ("PQ_plus_Vref", "tab:green")]:
    e2_axes[0].plot(
        e2_hours, e2_predictions[name][:, e2_worst],
        color=colour, label=MODEL_LABELS[name], alpha=0.8
    )
    e2_axes[1].plot(
        e2_hours, e2_errors[name][:, e2_worst],
        color=colour, label=MODEL_LABELS[name], linewidth=0.8
    )
e2_axes[0].set_ylabel("Voltage (V)")
e2_axes[0].set_title(f"Exercise: worst P/Q-only customer {e2_customer}")
e2_axes[0].legend()
e2_axes[1].axhline(0, color="black", linestyle="--")
e2_axes[1].set_xlabel("Hours from start of test period")
e2_axes[1].set_ylabel("Estimate - reference (V)")
e2_axes[1].set_title("Voltage residuals")
e2_axes[1].legend()
e2_fig.tight_layout()
plt.show()
```

Recorded output: customer 5. RMSE: 0.752155 V (P/Q only), 0.032014 V (P/Q + Vref).



Reading the plot. Vref reduces the voltage mismatch (top) and residuals (bottom) for this customer. Both models are shown for the same customer, selected using P/Q-only RMSE.